

2. SJF has the same turnaround time as FIFO when all of the jobs are the same size or the jobs monotonically increase in size (e.g., the shortest job arrives first, then the 2nd shortest job, then the 3rd and so on).
3. SJF delivers the same response times as RR when the scheduling quantum of RR is larger than the largest job to be serviced (so there is no preemption in RR) and when the jobs arrive in order of increasing job size.
4. As job lengths increase average response time goes up. If the jobs are sorted in increasing job size order, then the n -th job's response time is equal to the sum of the execution time of all the previous $n-1$ jobs plus that of the current job. Increasing the size of any job thus increases the response time for it and all larger jobs.
5. With RR, as quantum lengths increase response time increases because the time a process must wait for its next time slice (in the worst case) is proportional to the number of other processes and the maximum amount of time they can execute. Thus on a system with n processes and a quantum size of Q a process will have to wait for $Q(n-1)$ to respond to a request (in the worst case).
6. If you define efficiency as the ratio between the amount of CPU time spent on OS services versus time spent running userspace programs, preemptive schedulers are always less efficient because they switch between processes more often and hence run the scheduler and do context switches more often. Time spent running the scheduler and doing context switches is pure overhead. If you define efficiency in terms of response time - how long it takes the system to respond to input - preemptive schedulers are always more efficient than non-preemptive schedulers because preemptive schedulers can quickly switch to the process/thread that needs to process the input.
7. No, the problem will not occur with round-robin scheduling because processes can only use a certain amount (a ratio, relative to other processes) of time before it is forced to wait for the next process.

8. The four Gantt charts are

0				6	7		9			12				17
+	+	+	+	+	+	+	+	+	+	+	+	+	+	+
		P1			P2		P3			P4			P5	
+	+	+	+	+	+	+	+	+	+	+	+	+	+	+

FCFS

0	1		3		6				11				17
+	+	+	+	+	+	+	+	+	+	+	+	+	+
	P2		P3		P4			P5			P1		
+	+	+	+	+	+	+	+	+	+	+	+	+	+

SJF

0	1				6				12			15		17
+	+	+	+	+	+	+	+	+	+	+	+	+	+	+
	P2			P5			P1			P4		P3		
+	+	+	+	+	+	+	+	+	+	+	+	+	+	+

Priority

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
+	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+
	P1		P2		P3		P4		P5		P1		P4		P5		P1
+	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+	+

RR

(b) Turnaround time:

		FCFS		SJF		Priority		RR
P1		6		17		12		17
P2		7		1		1		2
P3		9		3		17		7
P4		12		6		15		11
P5		17		11		6		16
Average		10.2		7.6		10.2		10.6

(c) Waiting time:

		FCFS		SJF		Priority		RR
P1		0		11		6		11
P2		6		0		0		1
P3		7		1		15		5
P4		9		3		12		8
P5		12		6		1		11
Average		6.8		4.2		6.8		7.2

(d) Shortest Job First has the minimal average waiting time.

9. If we take all four previous run times into consideration, the prediction is $((((40 + 20) / 2 + 40) / 2 + 15) / 2 = ((30 + 40) / 2 + 15) / 2$

$$= (35 + 15) / 2 = 25$$

Or if take only two previous run times into consideration, the prediction is $(40 + 15) / 2 = 27.5$

10. **True.** Just before the user's time-slice expires, he or she can execute a dummy I/O request that makes the process appear to be "I/O Bound." Doing so allows the user to keep the task in the upper tier of the multi-level feedback scheduler, ensuring that he gets more CPU time than a CPU-intensive process should.